

Labor Verteilte Systeme

Aufgabe 3: Kafka und Eventgetriebene Kommunikation

Linetris-Client

Kurs: TINF23B

Dozent: Prof. Dr. rer. nat. Alexander Auch

1 Aufgabe 3a: Fragen zu Kafka und Partitionen

1.1 Datenorganisation in den Topics

Die Kommunikation läuft über zwei Topics: `game-requests` (Client → Game-Master) und `game-events` (Game-Master → Clients). Ein Topic ist in Partitionen unterteilt; jede Partition ist ein append-only Log mit monoton steigenden Offsets. Eine Reihenfolgegarantie besteht nur innerhalb einer Partition, nicht topicweit.

Die Nachrichten sind JSON-serialisiert. Requests enthalten genau eine fachliche Aktion (`newGame`, `move`). Events werden als `GameEventEnvelopeDTO` mit den Feldern `gameId`, `state` (`OK`/`ERROR`) und `actions` übertragen. Ein einzelnes Kafka-Event kann mehrere fachliche Aktionen enthalten (z. B. `move` + `deleteBottomRow`); diese werden im Client sequenziell verarbeitet.

1.2 Partitionierung

Die Zielpartition wird über den Kafka-Key bestimmt: $\text{partition} = \text{hash}(\text{key}) \bmod \text{Partitionsanzahl}$. Im Client wird konsequent die `gameId` als Key verwendet:

```
producer.send(dto, command.getGameId().toString());
```

Dadurch landen alle Events eines Spiels in derselben Partition, die Reihenfolge bleibt erhalten und der Spielzustand kann deterministisch rekonstruiert werden. Neben Load-Balancing dient die Partitionierung damit auch der Sicherstellung kausaler Ordnung pro Spiel.

1.3 Schema mit max. 10 Consumern; Vermeidung von Mehrfachempfang

Die maximale Parallelität innerhalb einer Consumer Group entspricht der Anzahl der Partitionen. Für bis zu 10 parallel arbeitende Consumer werden daher **mindestens 10 Partitionen** benötigt; überzählige Consumer bleiben sonst inaktiv.

Kafka liefert standardmäßig *at-least-once*. Um Mehrfachverarbeitung zu vermeiden, wird Auto-Commit deaktiviert, nach erfolgreicher Verarbeitung explizit committet und zusätzlich defensiv über das Tripel `topic-partition-offset` dedupliziert:

```
ENABLE_AUTO_COMMIT = false
consumer.commitSync();
recordId = topic + "-" + partition + "-" + offset
if (!processedRecords.add(recordId)) continue;
```

1.4 Standalone Consumer (ohne Gruppe)

Ein Standalone Consumer nutzt `assign()` statt `subscribe()`: keine automatische Partition-Zuweisung, kein Rebalancing, kein Failover, aber volle Kontrolle über Partitionen und Offsets. Geeignet für Debugging einzelner Partitionen, Replay zur Zustandsrekonstruktion sowie Monitoring von Event-Strömen. Für den Spielclient sind dagegen Consumer Groups sinnvoll, da mehrere Clients unabhängig den vollständigen Stream konsumieren sollen.

2 Aufgabe 3b: Linetris-Client

2.1 Architektur

Der Client ist gleichzeitig Producer und Consumer und folgt einer eventgetriebenen Projektion:

- **Producer:** sendet Requests (`newGame`, `move`) an `game-requests`. Ein Request ist nur eine Absicht, keine Zustandsänderung.
- **Consumer:** liest bestätigte Events aus `game-events`. Nur Events mit `state = OK` werden auf das lokale Modell angewendet; `ERROR`-Events werden nur angezeigt.
- **Game-Master als Single Source of Truth:** Zug-Validierung, Gewinnerkennung und `deleteBottomRow` liegen ausschließlich beim Game-Master. Der Client enthält keine Spiellogik, sondern projiziert nur den bestätigten Event-Strom.
- **Nebenläufigkeit:** getrennter `console-input`-Thread und `game-event-consumer`-Thread, damit blockierende Eingaben und Kafka-Polling sich nicht gegenseitig blockieren.

Paketstruktur.

Paket	Verantwortung
<code>aufgabe3.bootstrap</code>	Composition Root, Verdrahtung, Einstiegspunkt.
<code>aufgabe3.client</code>	Client-Steuerung, Konsoleneingabe, Spielkontext, Thread-Start.
<code>aufgabe3.application.usecase</code>	UseCases: Spiel starten, Zug senden, Event anwenden.
<code>aufgabe3.domain.command</code>	Commands (<code>StartGameCommand</code> , <code>PlaceMoveCommand</code>).
<code>aufgabe3.domain.event</code>	Events (<code>GameStartedEvent</code> , <code>MovePlacedEvent</code> , <code>BottomRowDeletedEvent</code> , <code>GameWonEvent</code>).
<code>aufgabe3.domain.model</code>	Modell (<code>GameState</code> , <code>Board</code> , <code>Player</code> , <code>GameStatus</code>).
<code>aufgabe3.domain.service</code>	<code>GameStateProjector</code> (Event-Projektion).
<code>aufgabe3.infrastructure.kafka</code>	Producer, Consumer, Kafka-Konfiguration.
<code>aufgabe3.infrastructure.repository</code>	In-Memory-Speicher des projizierten Zustands.
<code>aufgabe3.messaging.dto</code>	DTOs für die JSON-Kommunikation.
<code>aufgabe3.messaging.mapper</code>	Transformation <code>DTO ↔ Command/Event</code> .

2.2 Implementierung

gameId als Kafka-Key. Alle Requests eines Spiels werden mit derselben `gameId` als Key gesendet und landen so in einer Partition (kausale Ordnung pro Spiel):

```
1 producer.send(dto, command.getGameId().toString());
```

Eigene Consumer Group pro Client-Start. Damit jede Instanz den vollständigen Event-Stream erhält (Publish-Subscribe), erhält jeder Start eine eindeutige Group-ID. Eine geteilte Group würde die Partitionen aufteilen, sodass ein Client nur einen Teil der Events sähe:

```
1 props.put(ConsumerConfig.GROUP_ID_CONFIG, "game-client-" + UUID.randomUUID());
```

state zuerst auswerten. Der Consumer filtert auf die aktuelle gameId, behandelt ERROR vor actions und wendet nur OK-Events an:

```
1 if (currentGameId == null || !currentGameId.equals(dto.getGameId())) {
2     continue;
3 }
4 if ("ERROR".equals(dto.getState())) {
5     eventMapper.printError(dto);
6     continue;
7 }
8 List<GameEvent> events = eventMapper.map(dto);
9 for (GameEvent event : events) {
10     useCase.execute(event);
11 }
```

Getrennte Threads, thread-sicherer Kontext. Der CountdownLatch stellt sicher, dass erst nach gesetzter gameId verarbeitet wird:

```
1 this.consumerThread = new Thread(consumerLoop, "game-event-consumer");
2 this.inputThread = new Thread(inputController, "console-input");
3
4 private final AtomicReference<UUID> currentGameId = new AtomicReference<>();
5 private final AtomicReference<Player> localPlayer = new AtomicReference<>();
6 private final CountdownLatch gameIdLatch = new CountdownLatch(1);
```

Spielstart-Synchronisation zweier Instanzen. Die beiden Instanzen tauschen die gameId über ein eigenes Topic (nicht game-requests) aus; die Instanz von Spieler 1 initiiert das Spiel über den newGame-Request. Jedes neue Spiel erhält eine neue UUID.

2.3 Request- und Event-Formate

Der Client sendet ausschließlich Commands (Absichten):

```
1 {
2     "type": "newGame",
3     "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6",
4     "client1": { "name": "Client" },
5     "player1": { "name": "Player1" },
6     "client2": { "name": "Client" },
7     "player2": { "name": "Player2" }
8 }
```

```
1 {
2     "type": "move",
3     "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6",
4     "player": "PLAYER1",
5     "column": 1
6 }
```

Antworten aus game-events (bestätigt / Fehler / mehrere Aktionen / Gewinn):

```
1 {
2     "timeStamp": 1746130002467,
3     "actions": [ { "type": "move", "player": "PLAYER1", "column": 1 } ],
4     "state": "OK",
5     "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
6 }
```

```

1 {
2   "timeStamp": 1746130691239,
3   "actions": [ { "type": "move", "player": "PLAYER1", "column": 11 } ],
4   "state": "ERROR",
5   "message": "Illegal column number: 11, max=8",
6   "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
7 }

```

```

1 {
2   "timeStamp": 1746130485596,
3   "actions": [
4     { "type": "move", "player": "PLAYER1", "column": 8 },
5     { "type": "deleteBottomRow", "row": 8 }
6   ],
7   "state": "OK",
8   "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
9 }

```

```

1 {
2   "timeStamp": 1746130584078,
3   "actions": [
4     { "type": "move", "player": "PLAYER2", "column": 1 },
5     { "type": "winAction", "player": "PLAYER2" }
6   ],
7   "state": "OK",
8   "gameId": "a17c0b2f-1263-445d-b2bc-87ad89e2b3a6"
9 }

```

2.4 Tests

Getestet wird die Client-Verantwortung (korrekte, deterministische Event-Projektion), nicht die Spiellogik des Game-Masters.

Test	Prüft
GameStartedEvent	Zustand erst nach bestätigtem Start; Dimensionen übernommen; Board leer; Status RUNNING .
MovePlacedEvent	Stein in angegebener Spalte; 1-basiert → 0-basiert; tiefste freie Position; korrektes Stapeln.
ErrorHandlingTest	ERROR-Event erzeugt kein Domain-Event: <code>mapper.map(ERROR)</code> → <code>[]</code> .
RowDeletionTest	Unterste Reihe geleert, darüberliegende Reihen rücken nach unten.
WinConditionTest	Status WON , Gewinner aus Event; keine lokale Board-Analyse.
DeterminismTest	Gleiche Event-Sequenz → gleicher Zustand: <code>fold(events,A) == fold(events,B)</code> .
OrderingTest	Zug vor Start erzeugt keinen Zustand (Reihenfolge ist kausal relevant).
DeduplicationTest	Zwei gleiche OK-Events = zwei Fakten (keine inhaltliche Deduplizierung im Projektor).
ChaosIntegrationTest	Längere Sequenz inkl. Reihelöschung, Gewinn und Replay gegen zweites Repository.

Testausgaben (Auszüge). Die konkreten Beispielausgaben der Tests bleiben unverändert erhalten:

=== EVENT APPLIED: GameStartedEvent ===

GameId: c45d77a2-3712-471e-bc33-f3b45aa0fd6e

Status: RUNNING

Board:

```
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
```

MovePlacedEvent (zwei Zuege in derselben Spalte)

Board:

```
. . . . .
. . . . .
. . . . .
. . . . .
X . . . .
X . . . .
```

ErrorHandlingTest:

mapper.map(ERROR_EVENT) -> []

Falsches Verhalten waere:

state = ERROR, actions = [{type: "move", column: 1}]

-> MovePlacedEvent wird trotzdem erzeugt

-> lokales Board enthaelt einen abgelehnten Stein

RowDeletionTest:

Vorher:

```
. . . .
. . . .
. . . .
X X X X
```

Nach BottomRowDeletedEvent:

```
. . . .
. . . .
. . . .
. . . .
```

WinConditionTest:

=== EVENT APPLIED: GameWonEvent ===

Status: WON

WINNER: PLAYER1

DeterminismTest:

state = fold(orderedEvents)

fold(events, repositoryA) == fold(events, repositoryB)

OrderingTest:

Falsche Reihenfolge:

MovePlacedEvent(gameId, PLAYER1, column=1)

GameStartedEvent(gameId, rows=6, cols=6)

Erwarteter Zustand:

Anzahl Steine in column=1: 0

```
DeduplicationTest:
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER1, column=1)
Erwartung: 2 Steine in column=1

ChaosIntegrationTest (zentrale Assertion):
serializeBoard(state1) == serializeBoard(state2)
```

2.5 Nachweis des erfolgreichen Spiels

Verwendete gameId:

03704771-49c7-408b-aec8-6e186e17f69d

Chronologische Eventliste (realer Verlauf):

```
1. Spielstart:
GameStartedEvent(gameId, rows=6, cols=8)

2. Erste Zugsequenz:
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=2)
MovePlacedEvent(gameId, PLAYER1, column=3)
MovePlacedEvent(gameId, PLAYER2, column=4)
MovePlacedEvent(gameId, PLAYER1, column=5)
MovePlacedEvent(gameId, PLAYER2, column=6)
MovePlacedEvent(gameId, PLAYER1, column=7)
MovePlacedEvent(gameId, PLAYER2, column=8)

3. Reihenlöschung:
BottomRowDeletedEvent(gameId)

4. Vertikale Aufbauphase:
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=1)
MovePlacedEvent(gameId, PLAYER1, column=1)
MovePlacedEvent(gameId, PLAYER2, column=1)
MovePlacedEvent(gameId, PLAYER1, column=1)

5. Spielgewinn:
GameWonEvent(gameId, PLAYER1)
```

Finaler Zustand:

```
Status: WON
Winner: PLAYER1
```

Konsolenausgaben (Auszug).

```
=== EVENT APPLIED: GameStartedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . .
```

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

=== EVENT APPLIED: MovePlacedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . .
. . . . .
. . . . .
. . . . .
. 0 . . . . .
X 0 X 0 X 0 X 0

=== EVENT APPLIED: BottomRowDeletedEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: RUNNING
Board:
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. 0 . . . . .

=== EVENT APPLIED: GameWonEvent ===
GameId: 03704771-49c7-408b-aec8-6e186e17f69d
Status: WON
WINNER: PLAYER1
Board:
. . . . .
X . . . . .
0 . . . . .
X . . . . .
0 . . . . .
X 0 . . . . .

```

Der Gewinn wird nicht aus dem Board berechnet, sondern erst nach dem `GameWonEvent` gesetzt; die Reihenlöschung erfolgt erst nach dem `BottomRowDeletedEvent`.

3 Probleme und Lösungen

- **ERROR-Events:** werden nur ausgegeben, nicht angewendet (`continue` vor dem Mapping).
- **Synchronisation mehrerer Clients:** `gameId` als Partition-Key, lokale Filterung nach `gameId`, Zustand nur aus bestätigten Events.
- **Asynchrone Verarbeitung:** getrennter Input- und Consumer-Thread, Synchronisation über `GameContext` und `CountDownLatch`.
- **Event-Reihenfolge:** deterministische Partitionierung über `gameId`, sequenzielle Verarbeitung, keine lokale Validierung.

- **Mehrfachzustellung:** Offset-Deduplizierung und manuelles Commit nach Verarbeitung.
- **Initialisierung:** Zustand entsteht nur durch `GameStartedEvent`; vorherige Events werden ignoriert.

4 Fazit

Der lokale Spielzustand entsteht ausschließlich durch sequenzielle Projektion der vom Game-Master bestätigten Events. Die `gameId` als Kafka-Key sichert die kausale Reihenfolge pro Spiel innerhalb einer Partition; eigene Consumer Groups erlauben den parallelen Betrieb mehrerer Instanzen. Damit erfüllt der Client die Anforderungen der Aufgabe als reproduzierbares, eventgetriebenes Read-Model ohne lokale Spiellogik.